绘制箭头

绘制箭头
- 例程简介
- API 文档
- 示例代码
- 例程运行效果

例程简介

本节我们介绍绘制线条的draw_arrow()方法

API 文档

```
image.draw_arrow(x0, y0, x1, y1[, color[, thickness=1]])
```

在图像上绘制从 `(x0, y0)` 到 `(x1, y1)` 的箭头。参数可以分别传入 `x0, y0, x1, y1`，也可以作为元组 `(x0, y0, x1, y1)` 一起传递。

- **color**: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- **thickness**: 控制箭头线条的像素宽度，默认为 1。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

示例代码

```
# Import required modules
# 导入所需的模块
import time, os, urandom, sys, math

# Import display and media related modules
# 导入显示和媒体相关模块
from media.display import *
from media.media import *

# Define display resolution constants
# 定义显示分辨率常量
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480

def display_test():
    """
    Function to test display functionality
    测试显示功能的函数
    """

    # Create main background image with white color
```

```
    # 创建白色背景的主图像
    img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
    img.clear()
    img.draw_rectangle(0, 0, DISPLAY_WIDTH, DISPLAY_HEIGHT,color=
(255,255,255),fill=True)

    # Initialize display with ST7701 driver
    # 使用ST7701驱动初始化显示器
    Display.init(Display.ST7701, width = DISPLAY_WIDTH, height = DISPLAY_HEIGHT,
to_ide = True)
    # Initialize media manager
    # 初始化媒体管理器
    MediaManager.init()

    try:
        # 中央主要箭头，象征前进方向
        img.draw_arrow(320, 200, 400, 200, color=(0, 191, 255), thickness=5)  #
标准天蓝色

        # 辅助箭头，增加层次感
        img.draw_arrow(300, 180, 380, 180, color=(135, 206, 235), thickness=3)
 # 浅天蓝色
        img.draw_arrow(340, 220, 420, 220, color=(135, 206, 235), thickness=3)
 # 浅天蓝色

        # 对角线箭头，增加动感
        img.draw_arrow(250, 150, 350, 250, color=(0, 191, 255), thickness=3)  #
标准天蓝色
        img.draw_arrow(350, 150, 450, 250, color=(0, 191, 255), thickness=3)  #
标准天蓝色

        # 反向箭头，增加对比
        img.draw_arrow(400, 200, 320, 200, color=(173, 216, 230), thickness=3)
 # 最浅天蓝色
        img.draw_arrow(380, 180, 300, 180, color=(173, 216, 230), thickness=2)
 # 最浅天蓝色
        img.draw_arrow(420, 220, 340, 220, color=(173, 216, 230), thickness=2)
 # 最浅天蓝色

        # 垂直箭头，增加立体感
        img.draw_arrow(320, 150, 320, 250, color=(0, 191, 255), thickness=3)  #
标准天蓝色
        img.draw_arrow(400, 150, 400, 250, color=(0, 191, 255), thickness=3)  #
标准天蓝色

        # 点缀箭头，增加细节
        img.draw_arrow(300, 220, 310, 230, color=(135, 206, 235), thickness=2)
 # 浅天蓝色
        img.draw_arrow(330, 170, 340, 180, color=(135, 206, 235), thickness=2)
 # 浅天蓝色
        # Update display with background image
        # 更新显示背景图像
        Display.show_image(img)
        while True:
            time.sleep(2)

    except KeyboardInterrupt as e:
        print("user stop: ", e)
```

```
    except BaseException as e:
        print(f"Exception {e}")

    # Cleanup and deinitialize display
    # 清理并反初始化显示器
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
    # Release media resources
    # 释放媒体资源
    MediaManager.deinit()

if __name__ == "__main__":
    # Enable exit points and run display test
    # 启用退出点并运行显示测试
    os.exitpoint(os.EXITPOINT_ENABLE)
    display_test()
```

例程运行效果

